Concept-to-Animation: LLM-Generated Manim Explainer Videos

CSE 434 Midterm Check-in Report · November 13, 2026

Jonah Femiani Koray Bayram

Team: Concept-to-Animation **Roles:** *Jonah* — baseline pipeline (LLM client, planner, code generation, render harness, logging, ablation runner), repair and layout loops, analysis, demo. *Koray* — few-shot curation, RAG corpus and retriever, narration. *Both* — human evaluation and reports.

I. Project Overview

Given a short description of a math or CS concept (e.g. "binary search"), our system writes a short explainer video in the style of 3Blue1Brown by having an LLM plan a storyboard and then write Manim Community Edition (ManimCE) code for each scene. LLM-written animation code fails in two characteristic ways: it calls *hallucinated or outdated Manim APIs* (the code crashes), and it produces *bad layouts* (objects overlap or leave the frame). Our research question is: **which generative techniques make LLM-written animation code render reliably and look clear?** We answer it with a cumulative ablation that starts from a zero-shot baseline and adds one technique at a time: error-driven self-repair, few-shot examples, retrieval-augmented generation (RAG) over the ManimCE docs, and a layout-repair loop driven by a bounding-box checker. This connects directly to course topics on prompting, in-context learning, RAG, iterative self-refinement, and LLM code generation.

II. Dataset

Concepts. We curated 6 concepts in three difficulty tiers of two each, chosen by how directly the idea maps to pictures: *concrete* (Pythagorean theorem, derivative as slope), *structural* (matrix multiplication, binary search), and *abstract* (recursion, Big-O notation). Each has a one-sentence prompt; the tiers let us ask whether performance drops as concepts get less visual.

Storyboards. For each concept the planner produces a storyboard of 3–4 scenes (20 scenes in total), each with a title, a visual description and narration. Storyboards are schema-validated and cached (one batch, one model), so every condition generates code for exactly the same scenes.

Retrieval corpus (planned): the ManimCE docs and example gallery (MIT license), pinned to our render version (v0.22.1), chunked and embedded for RAG.

Ethics. The data contains no personal information. The main risk is that a fluent, polished video can teach *wrong* math; render success does not catch this, so we add human ratings of mathematical correctness. Generated videos will be labeled as AI-generated.

III. Methods

Figure 1 shows the current system (v1). A **planner** turns the concept into a storyboard. A **code generator** writes one ManimCE `Scene` subclass per storyboard scene; its prompt states the pinned Manim version, forbids the older ManimGL library, and gives the frame bounds ($x \in [-7.1, 7.1]$, $y \in [-4, 4]$). Post-processing is deliberately minimal (extract the code block, rename the class) so it cannot silently fix the errors we are measuring. Each scene is **rendered** in a separate subprocess with a 120 s timeout; we check that the expected video file was actually written, because Manim exits successfully even when the named scene does not exist. Failures are **classified** from the traceback into `hallucinated_api`, `bad_signature`, `latex`, `timeout`, `syntax` or `other`. Manim's `Mobject` invents any `get_*` method on demand, so a hallucinated call like `axes.get_secant_line(...)` surfaces as a `TypeError`; our classifier still labels it a hallucinated API. Every attempt is logged (tokens, latency, error, code) and responses are cached.

Experimental design. The full study is a cumulative ablation over six conditions: `zero_shot`, `zero_shot_repair`, `few_shot`, `rag`, `rag_repair` and `rag_repair_layout`. Temperature is fixed

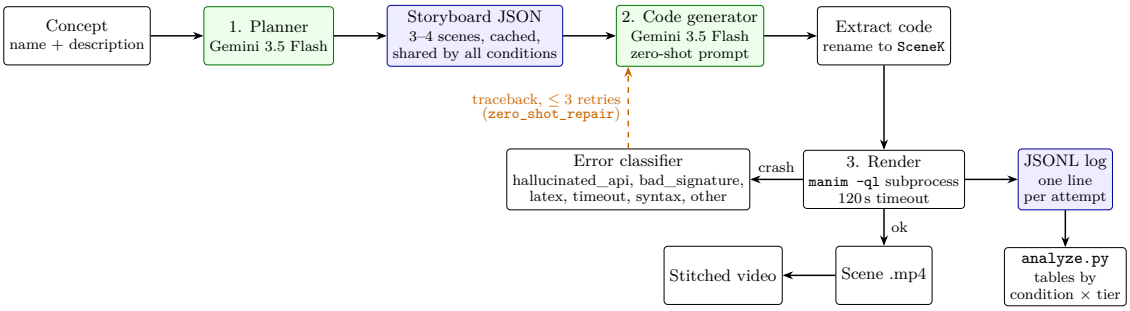


Figure 1: System design v1. Green: LLM calls; blue: stored artifacts. Solid path: the zero-shot baseline. Dashed: the error-repair loop (`zero_shot_repair`), which feeds the traceback back to the code generator. Few-shot, RAG and layout repair will attach to the code generator later.

at 0.2, conditions without repair get exactly one render attempt, and repair conditions get up to three extra attempts. Per-scene metrics, aggregated by condition and tier: render success, first-try success, repair iterations, layout-defect rate, error types, and two human scores (failed scene = 0, so they reflect the whole pipeline).

Changes since the proposal (approved by Dr. Zhang). (1) *Gemini 3.5 Flash instead of GPT + an open model:* we tried Qwen2.5-Coder (7B and 3B, 4-bit via Ollama), but its Manim code was too poor to make the comparison informative; open models stay available behind our LLM interface. (2) *No LangChain:* plain Python lets us log every prompt, reply and token count exactly. (3) *6 concepts instead of 40,* so every condition can be run and every video hand-rated.

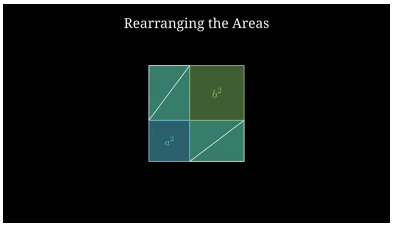
IV. Preliminary Results

Table 1 reports both conditions on all 20 scenes. Zero-shot renders 70% of scenes, roughly evenly across tiers; error-driven repair raises this to **100%**, needing 0.45 extra attempts per scene on average. Structural scenes (matrix grids, arrays) needed the most help (0.83 repairs per scene), so at this stage difficulty tracks how much layout code a scene needs, not how abstract the concept is. Repair costs 2.3× the tokens per scene (4.3k vs. 1.9k). Every call reported `gemini-3.6-flash`, so `gemini-3.5-flash` is an alias; we log the version to detect drift.

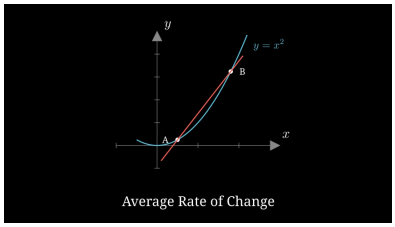
	Automatic			Human (0–5; failed scene = 0)	
Condition	Render success	First-try success	Avg. repairs	Correctness	Clarity
`zero_shot`	70% (14/20)	70%	–	3.20	2.80
concrete (7)	71% (5/7)	71%	–	3.57	3.07
structural (6)	67% (4/6)	67%	–	2.83	2.33
abstract (7)	71% (5/7)	71%	–	3.14	2.93
`zero_shot_repair`	**100% (20/20)**	70%	0.45	**4.60**	**3.92**
concrete (7)	100% (7/7)	71%	0.29	4.93	4.29
structural (6)	100% (6/6)	67%	0.83	4.25	3.08
abstract (7)	100% (7/7)	71%	0.29	4.57	4.29

Table 1: Gemini 3.5 Flash, all 20 scenes (scenes per tier in parentheses). For `zero_shot_repair`, render minus first-try success is what repair adds; avg. repairs is over scenes that eventually rendered. Human scores: mean of two raters over all 20 scenes; rendered scenes rated 1–5 blind to condition, failed scenes scored 0.

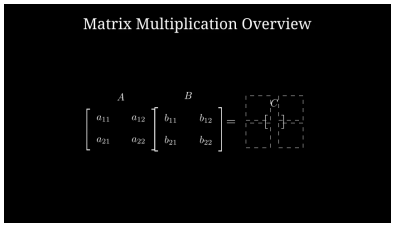
Qualitative examples. *Success:* all four Pythagorean-theorem scenes rendered first try;



(a) Pythagorean theorem, scene 3: correct and clear.



(b) Derivative as slope, scene 1: correct; scenes 2–3 needed repair.



(c) Matrix mult., scene 1: rendered, but rated 2/5 correct.

Figure 2: Final frames of three zero-shot scenes that all rendered on the first try. (c) shows why render success alone is not enough: no operator between A and B, mismatched brackets, and an empty dashed grid for C.

scene 3 (Fig. 2a) rearranges four triangles in an $(a + b)$ square to expose a^2 and b^2, a clean, correct version of the classic proof. *Contrasting failure:* the derivative's static scene 1 (Fig. 2b) is fine, but scenes 2 and 3, which must *animate* the secant approaching the tangent, crashed zero-shot on methods that sound right but do not exist (`axes.get_secant_line`, `get_tangent_line`). Given the traceback, repair replaced each with a small helper method that computes the line itself, and both scenes rendered on the second attempt. Repair is not always one step: in matrix multiplication scene 2, fixing an invalid keyword exposed an undefined color `CYAN` (which the model then defined itself), then a LaTeX error; it rendered on the fourth and last attempt.

Error types. All six zero-shot crashes are API misuse, never syntax: three `bad_signature`, two `hallucinated_api`, one misused `TransformMatchingTex`. Each error names the bad call, which is why repair works and what RAG should prevent up front.

Human ratings. Both authors rated the 20 distinct videos (repair's first-try successes are identical to zero-shot's) in shuffled order with the condition hidden. Repair raises correctness from 3.20 to 4.60 and clarity from 2.80 to 3.92, mostly by turning 0s into real videos; the repaired scenes themselves score as well as first-try successes (correctness 4.67 vs. 4.57), so repair does not degrade quality. Clarity trails correctness in every tier, and our notes point at layout, not math: labels hidden under shapes, arrows over numbers, overlapping grids, which is exactly what the planned layout checker targets. Structural scenes are the weakest on both (clarity 3.08 even after repair). Two rendered scenes have real content errors that no automatic metric caught: Fig. 2c, and a Big-O plot whose $n \log n$ curve is drawn wrong. Raters agreed closely on correctness (80% exact, 100% within one point, $r = 0.88$) but less on clarity (50% exact, 80% within one, $r = 0.63$), so we will write a clarity rubric before the final evaluation.

V. Next Steps

- **Few-shot and RAG** (Koray): 3–5 curated modern example scenes, and a retriever over the pinned ManimCE docs, checked on whether it returns the right classes.
- **Layout repair:** snapshot bounding boxes after each animation, flag off-frame objects and unintended overlaps, and describe them to the LLM in plain language. With render success at 100%, clarity, our lowest score, is now the target.
- **Evaluation and demo:** the full six-condition ablation by tier, cost per condition, a success/failure gallery, gTTS narration and a type-a-concept web demo.

Open questions for feedback. Are 20 scenes enough to separate conditions now that render success has saturated? Is a bounding-box check a fair proxy for clarity, or should clarity rest on human ratings? Should we add leave-one-out rows to check the ablation's stacking order?